

Dropout

2026 年 6 月 10 日

1 dropout (随机失活)

dropout[1] 是一种通过在正向传播过程中随机将一些输出激活值设为零来正则化神经网络的技术。在本练习中，你需要实现一个 dropout 层，并修改你的全连接网络以选择性地使用 dropout。

[1] Geoffrey E. Hinton et al, “Improving neural networks by preventing co-adaptation of feature detectors”, arXiv 2012

```
[2]: # 配置 matplotlib 中文字体
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False

# 配置单元格
import time # 导入时间模块, 用于计时
import numpy as np # 导入 numpy 库, 用于数值计算
import matplotlib.pyplot as plt # 导入 matplotlib 库, 用于绘图
from cs231n.classifiers.fc_net import * # 从 cs231n 的分类器模块导入全连接网络相关类/函数
from cs231n.data_utils import get_CIFAR10_data # 从数据工具模块导入获取 CIFAR-10 数据的函数
from cs231n.gradient_check import eval_numerical_gradient, \
    eval_numerical_gradient_array # 导入梯度检查相关函数
from cs231n.solver import Solver # 导入求解器类, 用于模型训练

# 设置 matplotlib 在 notebook 中内联显示
%matplotlib inline
plt.rcParams["figure.figsize"] = (10.0, 8.0) # 设置图表的默认大小
plt.rcParams["image.interpolation"] = "nearest" # 设置图像插值方式为最近邻
```

```
plt.rcParams["image.cmap"] = "gray" # 设置默认的图像颜色映射为灰度

# 加载自动重载扩展
%load_ext autoreload
# 设置自动重载模式，当导入的模块被修改时自动重新导入
%autoreload 2

def rel_error(x, y):
    """ 返回相对误差 """
    return np.max(np.abs(x - y) / (np.maximum(1e-8, np.abs(x) + np.abs(y)))) # 计算 x 和 y 之间的相对误差，避免除零
```

```
[3]: # 加载（预处理后的）CIFAR-10 数据
data = get_CIFAR10_data()
# 遍历数据字典中的所有键值对并打印
for k, v in list(data.items()):
    print(f"{k}: {v.shape}") # 打印每个数据集的名称及其形状
```

```
X_train: (49000, 3, 32, 32)
y_train: (49000,)
X_val: (1000, 3, 32, 32)
y_val: (1000,)
X_test: (1000, 3, 32, 32)
y_test: (1000,)
```

2 Dropout：前向传播

在文件 `cs231n/layers.py` 中，实现 dropout 的前向传播。由于 dropout 在训练模式和测试模式下的行为不同，要确保为两种模式都实现相应的操作。

完成实现后，运行下面的单元格来测试你的代码。

```
[4]: np.random.seed(231) # 设置随机种子，保证结果可复现
x = np.random.randn(500, 500) + 10 # 生成随机输入数据，均值约为 10，形状为 (500, 500)

# 测试不同的 dropout 概率 p
for p in [0.25, 0.4, 0.7]:
```

```

# 训练模式下的 dropout 前向传播 (p 为保留神经元的概率)
out, _ = dropout_forward(x, {'mode': 'train', 'p': p})
# 测试模式下的 dropout 前向传播
out_test, _ = dropout_forward(x, {'mode': 'test', 'p': p})

print('测试 p = ', p, ' 时的情况')
print('输入数据的均值: ', x.mean())
print('训练模式下输出的均值: ', out.mean())
print('测试模式下输出的均值: ', out_test.mean())
print('训练模式下输出中被设为 0 的比例: ', (out == 0).mean())
print('测试模式下输出中被设为 0 的比例: ', (out_test == 0).mean())
print() # 空行分隔不同 p 的测试结果

```

测试 p = 0.25 时的情况

输入数据的均值: 10.000207878477502

训练模式下输出的均值: 10.014059116977283

测试模式下输出的均值: 10.000207878477502

训练模式下输出中被设为 0 的比例: 0.749784

测试模式下输出中被设为 0 的比例: 0.0

测试 p = 0.4 时的情况

输入数据的均值: 10.000207878477502

训练模式下输出的均值: 9.977917658761159

测试模式下输出的均值: 10.000207878477502

训练模式下输出中被设为 0 的比例: 0.600796

测试模式下输出中被设为 0 的比例: 0.0

测试 p = 0.7 时的情况

输入数据的均值: 10.000207878477502

训练模式下输出的均值: 9.987811912159426

测试模式下输出的均值: 10.000207878477502

训练模式下输出中被设为 0 的比例: 0.30074

测试模式下输出中被设为 0 的比例: 0.0

3 Dropout: 反向传播

在文件 `cs231n/layers.py` 中, 实现 dropout 的反向传播。完成后, 运行以下单元格, 通过数值梯度检查来验证你的实现。

```
[5]: np.random.seed(231) # 设置随机种子, 确保结果可复现
x = np.random.randn(10, 10) + 10 # 生成随机输入数据, 均值约为 10, 形状为 (10, 10)
dout = np.random.randn(*x.shape) # 生成与 x 形状相同的随机输出梯度

# dropout 参数: 训练模式, 保留概率 p=0.2, 随机种子 123 (保证 dropout 掩码固定)
dropout_param = {'mode': 'train', 'p': 0.2, 'seed': 123}
out, cache = dropout_forward(x, dropout_param) # 执行 dropout 前向传播, 获取输出和缓存
dx = dropout_backward(dout, cache) # 执行 dropout 反向传播, 计算输入梯度 dx
# 用数值方法计算梯度作为参考 (数值梯度)
dx_num = eval_numerical_gradient_array(lambda xx: dropout_forward(xx, dropout_param)[0], x, dout)

# 误差应在 1e-10 左右或更小
print('dx 的相对误差: ', rel_error(dx, dx_num))
```

dx 的相对误差: 5.44560814873387e-11

3.1 内联问题 1:

如果在 dropout 层中, 不将通过反向 dropout 的数值除以 p , 会发生什么? 为什么会出现这种情况?

3.2 答案:

这会导致以下问题:

1. 在测试阶段, 所有神经元都参与前向传播, 不进行 dropout, 输出的期望值是原来的 1 倍。而在训练阶段, 如果不进行缩放, 输出的期望值只有原来的 p 倍。这种训练和测试之间的不一致会导致模型性能下降。
2. 反向传播时, 如果不除以 p , 反向传播通过 dropout 层的梯度也会缩小到原来的 p 倍。这会导致前面的层接收到的梯度信号变弱, 影响参数更新, 可能导致训练收敛变慢或不稳定。

Inverted Dropout 的原理: 通过将保留的神经元乘以 $1/p$ (等价于反向传播时除以 p), 使得前向传播的期望输出保持不变 ($p \times (1/p) = 1$), 从而保证训练和测试时激活值的尺度一致, 梯度也能

正确传播。

4 带 Dropout 的全连接网络

在文件 `cs231n/classifiers/fc_net.py` 中，修改你的实现以使用 `dropout`。具体来说，如果网络的构造函数收到的 `dropout_keep_ratio` 参数值不是 1，那么网络应该在每个 ReLU 非线性激活之后立即添加一个 `dropout` 层。完成后，运行以下代码通过数值梯度检查来验证你的实现。

```
[6]: np.random.seed(231) # 设置随机种子，确保结果可复现
# N: 批量大小, D: 输入维度, H1: 第一层隐藏层大小, H2: 第二层隐藏层大小, C: 类别数
N, D, H1, H2, C = 2, 15, 20, 30, 10
X = np.random.randn(N, D) # 生成随机输入数据
y = np.random.randint(C, size=(N,)) # 生成随机标签

# 测试不同的 dropout 保留比例
for dropout_keep_ratio in [1, 0.75, 0.5]:
    print('测试 dropout = ', dropout_keep_ratio, ' 时的情况')
    # 创建全连接网络模型
    model = FullyConnectedNet(
        [H1, H2], # 隐藏层大小列表
        input_dim=D, # 输入维度
        num_classes=C, # 类别数
        weight_scale=5e-2, # 权重初始化缩放因子
        dtype=np.float64, # 数据类型
        dropout_keep_ratio=dropout_keep_ratio, # dropout 保留比例
        seed=123 # 随机种子，保证 dropout 行为可复现
    )

    loss, grads = model.loss(X, y) # 计算损失和梯度
    print('初始损失: ', loss)

    # 相对误差应在 1e-6 左右或更小
    # 注意：当 dropout_keep_ratio=1 时，W2 的误差在 1e-5 量级左右是可接受的
    for name in sorted(grads):
        # 定义一个仅返回损失的函数，用于数值梯度计算
        f = lambda _: model.loss(X, y)[0]
        # 数值计算梯度
```

```

grad_num = eval_numerical_gradient(f, model.params[name],
↪ verbose=False, h=1e-5)
    # 打印梯度的相对误差
    print('%s 的相对误差: %.2e' % (name, rel_error(grad_num, grads[name])))
print() # 空行分隔不同 dropout 保留比例的测试结果

```

测试 dropout = 1 时的情况

初始损失: 2.300479089768492

W1 的相对误差: 1.03e-07

W2 的相对误差: 2.21e-05

W3 的相对误差: 4.56e-07

b1 的相对误差: 4.66e-09

b2 的相对误差: 2.09e-09

b3 的相对误差: 1.69e-10

测试 dropout = 0.75 时的情况

初始损失: 2.302371489704412

W1 的相对误差: 1.85e-07

W2 的相对误差: 2.15e-06

W3 的相对误差: 4.56e-08

b1 的相对误差: 1.16e-08

b2 的相对误差: 1.82e-09

b3 的相对误差: 1.48e-10

测试 dropout = 0.5 时的情况

初始损失: 2.30427592207859

W1 的相对误差: 3.11e-07

W2 的相对误差: 2.48e-08

W3 的相对误差: 6.43e-08

b1 的相对误差: 5.37e-09

b2 的相对误差: 1.91e-09

b3 的相对误差: 1.85e-10

5 正则化实验

作为一项实验，我们将在 500 个训练样本上训练两个双层网络：一个不使用 dropout，另一个使用 0.25 的保留概率。然后我们将可视化这两个网络随时间变化的训练准确率和验证准确率。

```
[9]: # 训练两个相同的网络，一个使用 dropout，一个不使用
np.random.seed(231) # 设置随机种子，确保实验可复现
num_train = 500 # 训练样本数量
small_data = {
    'X_train': data['X_train'][:num_train], # 取前 500 个训练样本
    'y_train': data['y_train'][:num_train],
    'X_val': data['X_val'], # 验证集数据
    'y_val': data['y_val'],
}

solvers = {} # 用于存储不同 dropout 设置的求解器
dropout_choices = [1, 0.25] # 测试两种 dropout 保留比例: 1 (无 dropout) 和 0.25
for dropout_keep_ratio in dropout_choices:
    # 创建全连接网络模型，包含一个 500 维的隐藏层
    model = FullyConnectedNet(
        [500],
        dropout_keep_ratio=dropout_keep_ratio # 设置 dropout 保留比例
    )
    print(dropout_keep_ratio) # 打印当前的 dropout 保留比例

    # 创建求解器用于训练模型
    solver = Solver(
        model, # 要训练的模型
        small_data, # 使用的数据集
        num_epochs=25, # 训练轮数
        batch_size=100, # 批大小
        update_rule='adam', # 使用 adam 优化算法
        optim_config={'learning_rate': 5e-4,}, # 优化器配置，学习率为 5e-4
        verbose=True, # 训练过程中打印详细信息
        print_every=100 # 每 100 次迭代打印一次信息
    )
    solver.train() # 训练模型
```

```
solvers[dropout_keep_ratio] = solver # 将训练好的求解器存入字典
print() # 空行分隔不同 dropout 设置的训练过程
```

1

(迭代 1 / 125) 损失: 7.856644

(Epoch 0 / 25) 训练准确率: 0.260000; 验证准确率: 0.184000

(Epoch 1 / 25) 训练准确率: 0.416000; 验证准确率: 0.258000

(Epoch 2 / 25) 训练准确率: 0.482000; 验证准确率: 0.276000

(Epoch 3 / 25) 训练准确率: 0.532000; 验证准确率: 0.277000

(Epoch 4 / 25) 训练准确率: 0.600000; 验证准确率: 0.271000

(Epoch 5 / 25) 训练准确率: 0.710000; 验证准确率: 0.298000

(Epoch 6 / 25) 训练准确率: 0.722000; 验证准确率: 0.282000

(Epoch 7 / 25) 训练准确率: 0.832000; 验证准确率: 0.255000

(Epoch 8 / 25) 训练准确率: 0.874000; 验证准确率: 0.267000

(Epoch 9 / 25) 训练准确率: 0.898000; 验证准确率: 0.277000

(Epoch 10 / 25) 训练准确率: 0.902000; 验证准确率: 0.260000

(Epoch 11 / 25) 训练准确率: 0.938000; 验证准确率: 0.266000

(Epoch 12 / 25) 训练准确率: 0.964000; 验证准确率: 0.298000

(Epoch 13 / 25) 训练准确率: 0.974000; 验证准确率: 0.304000

(Epoch 14 / 25) 训练准确率: 0.970000; 验证准确率: 0.323000

(Epoch 15 / 25) 训练准确率: 0.976000; 验证准确率: 0.301000

(Epoch 16 / 25) 训练准确率: 0.982000; 验证准确率: 0.302000

(Epoch 17 / 25) 训练准确率: 0.972000; 验证准确率: 0.301000

(Epoch 18 / 25) 训练准确率: 0.976000; 验证准确率: 0.299000

(Epoch 19 / 25) 训练准确率: 0.982000; 验证准确率: 0.308000

(Epoch 20 / 25) 训练准确率: 0.968000; 验证准确率: 0.304000

(迭代 101 / 125) 损失: 0.203952

(Epoch 21 / 25) 训练准确率: 0.990000; 验证准确率: 0.315000

(Epoch 22 / 25) 训练准确率: 0.992000; 验证准确率: 0.302000

(Epoch 23 / 25) 训练准确率: 0.968000; 验证准确率: 0.295000

(Epoch 24 / 25) 训练准确率: 0.984000; 验证准确率: 0.287000

(Epoch 25 / 25) 训练准确率: 0.982000; 验证准确率: 0.286000

0.25

(迭代 1 / 125) 损失: 17.318479

(Epoch 0 / 25) 训练准确率: 0.230000; 验证准确率: 0.177000

(Epoch 1 / 25) 训练准确率: 0.378000; 验证准确率: 0.243000

(Epoch 2 / 25) 训练准确率: 0.404000; 验证准确率: 0.254000
(Epoch 3 / 25) 训练准确率: 0.496000; 验证准确率: 0.276000
(Epoch 4 / 25) 训练准确率: 0.526000; 验证准确率: 0.300000
(Epoch 5 / 25) 训练准确率: 0.576000; 验证准确率: 0.301000
(Epoch 6 / 25) 训练准确率: 0.618000; 验证准确率: 0.296000
(Epoch 7 / 25) 训练准确率: 0.658000; 验证准确率: 0.307000
(Epoch 8 / 25) 训练准确率: 0.692000; 验证准确率: 0.316000
(Epoch 9 / 25) 训练准确率: 0.730000; 验证准确率: 0.297000
(Epoch 10 / 25) 训练准确率: 0.728000; 验证准确率: 0.304000
(Epoch 11 / 25) 训练准确率: 0.794000; 验证准确率: 0.316000
(Epoch 12 / 25) 训练准确率: 0.784000; 验证准确率: 0.275000
(Epoch 13 / 25) 训练准确率: 0.814000; 验证准确率: 0.310000
(Epoch 14 / 25) 训练准确率: 0.786000; 验证准确率: 0.324000
(Epoch 15 / 25) 训练准确率: 0.828000; 验证准确率: 0.332000
(Epoch 16 / 25) 训练准确率: 0.820000; 验证准确率: 0.300000
(Epoch 17 / 25) 训练准确率: 0.832000; 验证准确率: 0.292000
(Epoch 18 / 25) 训练准确率: 0.856000; 验证准确率: 0.337000
(Epoch 19 / 25) 训练准确率: 0.884000; 验证准确率: 0.330000
(Epoch 20 / 25) 训练准确率: 0.872000; 验证准确率: 0.317000
(迭代 101 / 125) 损失: 4.219688
(Epoch 21 / 25) 训练准确率: 0.886000; 验证准确率: 0.314000
(Epoch 22 / 25) 训练准确率: 0.878000; 验证准确率: 0.309000
(Epoch 23 / 25) 训练准确率: 0.878000; 验证准确率: 0.292000
(Epoch 24 / 25) 训练准确率: 0.898000; 验证准确率: 0.312000
(Epoch 25 / 25) 训练准确率: 0.900000; 验证准确率: 0.310000

```
[10]: # 绘制两个模型的训练准确率和验证准确率
train_accs = [] # 存储训练准确率
val_accs = [] # 存储验证准确率
for dropout_keep_ratio in dropout_choices:
    solver = solvers[dropout_keep_ratio]
    train_accs.append(solver.train_acc_history[-1]) # 取最后一轮的训练准确率
    val_accs.append(solver.val_acc_history[-1]) # 取最后一轮的验证准确率

plt.subplot(3, 1, 1) # 创建 3 行 1 列的子图, 当前绘制第 1 个
for dropout_keep_ratio in dropout_choices:
```

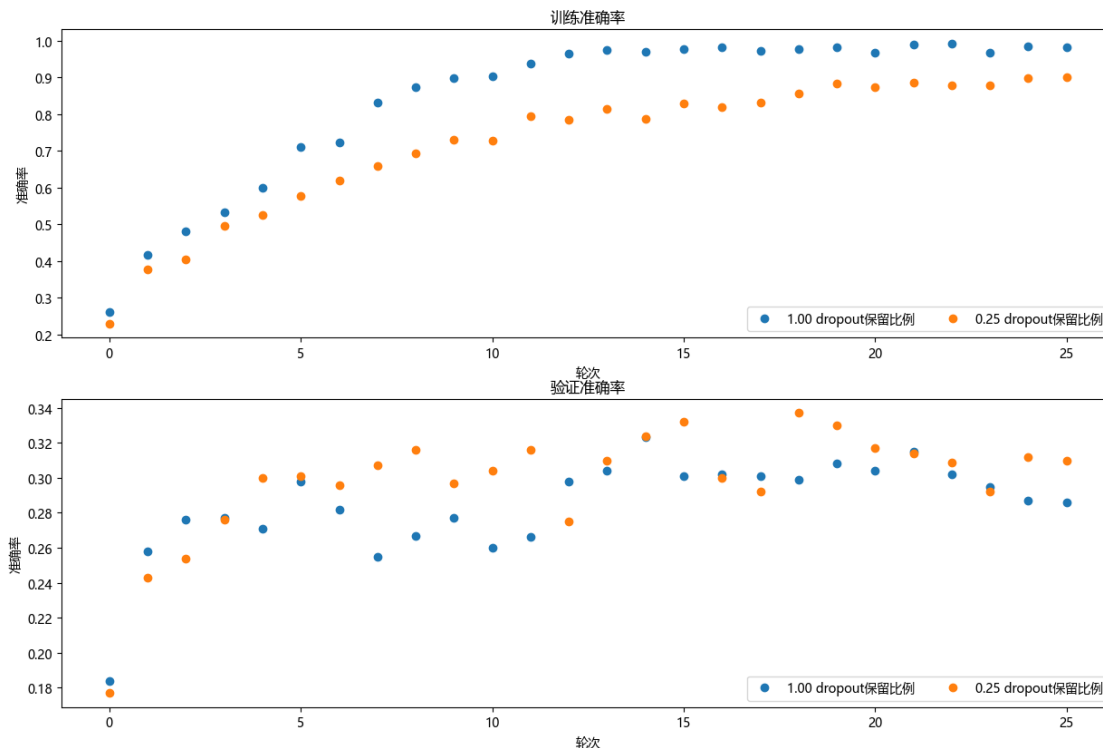
```

# 绘制训练准确率曲线, 标记为 'o', 标签为对应的 dropout 保留比例
plt.plot(
    solvers[dropout_keep_ratio].train_acc_history, 'o', label='%0.2f dropout
保留比例' % dropout_keep_ratio)
plt.title('训练准确率')
plt.xlabel('轮次')
plt.ylabel('准确率')
plt.legend(ncol=2, loc='lower right') # 图例显示在右下角, 2 列

plt.subplot(3, 1, 2) # 当前绘制第 2 个子图
for dropout_keep_ratio in dropout_choices:
    # 绘制验证准确率曲线, 标记为 'o', 标签为对应的 dropout 保留比例
    plt.plot(
        solvers[dropout_keep_ratio].val_acc_history, 'o', label='%0.2f dropout 保
留比例' % dropout_keep_ratio)
plt.title('验证准确率')
plt.xlabel('轮次')
plt.ylabel('准确率')
plt.legend(ncol=2, loc='lower right') # 图例显示在右下角, 2 列

plt.gcf().set_size_inches(15, 15) # 设置图表大小
plt.show() # 显示图表

```



5.1 内联问题 2:

比较使用 dropout 和不使用 dropout 时的验证准确率与训练准确率——你的结果说明了 dropout 作为正则化器有什么作用？

5.2 答案:

从实验结果可以看出:

训练准确率对比: - 不使用 dropout ($p=1.0$): 最终训练准确率达到约 98.2% - 使用 dropout ($p=0.25$): 最终训练准确率约为 90.0%

验证准确率对比: - 不使用 dropout ($p=1.0$): 最终验证准确率约为 28.6% - 使用 dropout ($p=0.25$): 最终验证准确率约为 31.0%

Dropout 作为正则化器的作用:

1. **减轻过拟合:** 不使用 dropout 的模型训练准确率很高 (98.2%), 但验证准确率较低 (28.6%), 训练和验证准确率之间存在较大差距, 这是典型的过拟合现象。而使用 dropout 的模型训练准确率较低 (90.0%), 但验证准确率更高 (31.0%), 泛化能力更强。

2. **缩小训练-验证差距：**dropout 通过随机丢弃神经元，阻止了神经元之间的共适应（co-adaptation），使得模型不会过度依赖某些特定的特征或神经元组合，从而提高了模型的泛化能力。
3. **效果总结：**虽然 dropout 降低了训练准确率，但它提高了验证准确率，说明 dropout 有效地作为正则化手段，帮助模型在未见数据上表现更好。这验证了 dropout 作为一种正则化技术的有效性。